



CODE 101

Introduction to Software Development and Careers in Technology

WiFi: GeoGuest / Welcome2Geo!

<https://github.com/DeltaVCode/code-101-prework>

Open Git Bash or Terminal and verify installs:

```
git --version  
atom --version
```

Join GitHub.com, verify email address, and sign in

<https://bit.ly/deltav-slack> to join <https://deltavcode.slack.com/>



Ice Breaker

- **Your name**
- **Why you're here/What you want to gain from this workshop**
- **Something interesting about yourself**

Goals

- Get to know the life of a professional software developer.
- Understand the frameworks of modern websites.
- Code a complete website using HTML and CSS.
- Deploy your website on the Internet.
- Figure out whether coding is for you.

Our Agenda

- Intros, Intro to the Modern Web and HTML
- Wireframing
- HTML and Team Coding
- How to Share and Deploy Code
- Team Coding
- Lunch and tips for learning to code
- Intro to CSS and JavaScript
- What is a "Software Developer"?
- Dinner
- Presentations
- Resources for Learning More and Survey

How You'll Learn

"Path" Learning:

- Leads you along
- Students are consumers of information
- Predictable outcomes
- Creates dependency
- The goal: an exchange of information

"Sandbox" Learning:

- Fosters exploration
- Students are co-creators of their learning experience
- Wide range of outcomes
- Creates autonomy
- The goal: learning and discovery

Skills For Sandbox Learning

- Generating and selecting **ideas**: what do you want to learn now?
- Planning your **learning**: managing scope, finding resources
- **Experimentation**: keeping track of what you've tried, what's worked, and what hasn't
- **Reflection**: pausing every so often to tally what you've learned, and what new questions you have
- Finding **help**!

Your Tools

- Your new friends
- Your instructor and TAs
- The Internet (Google, Mozilla MDN, Stack Overflow)
- Slack



Slack



The Modern Web

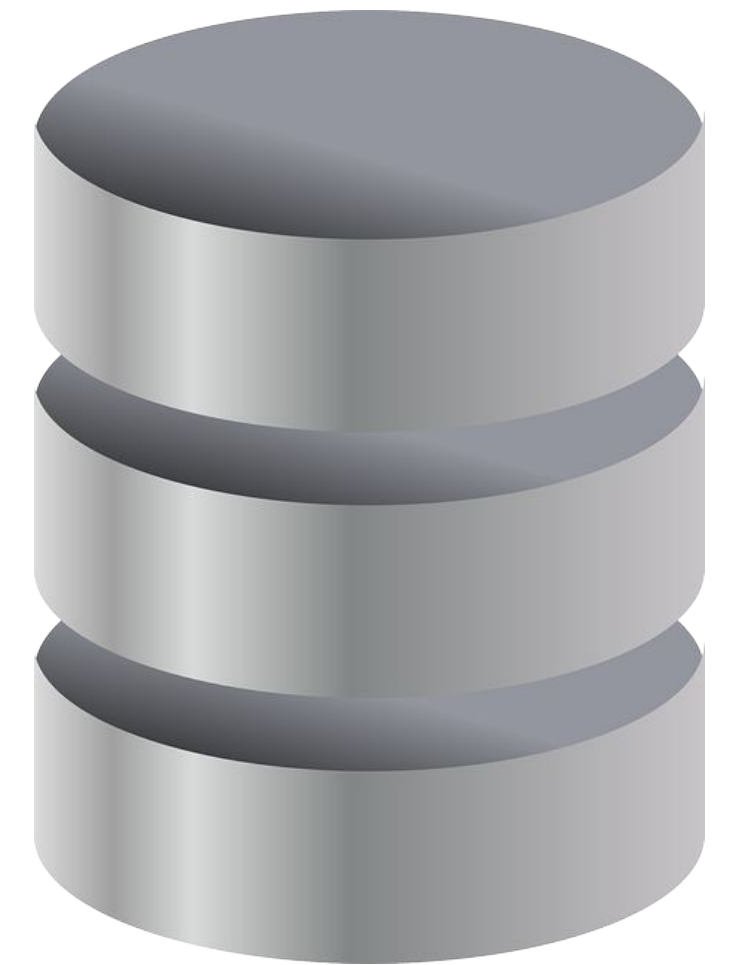
Web Applications



Client



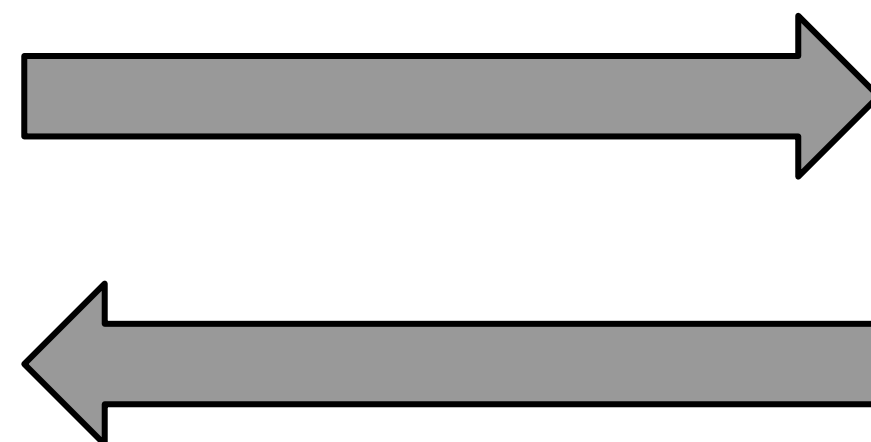
Server



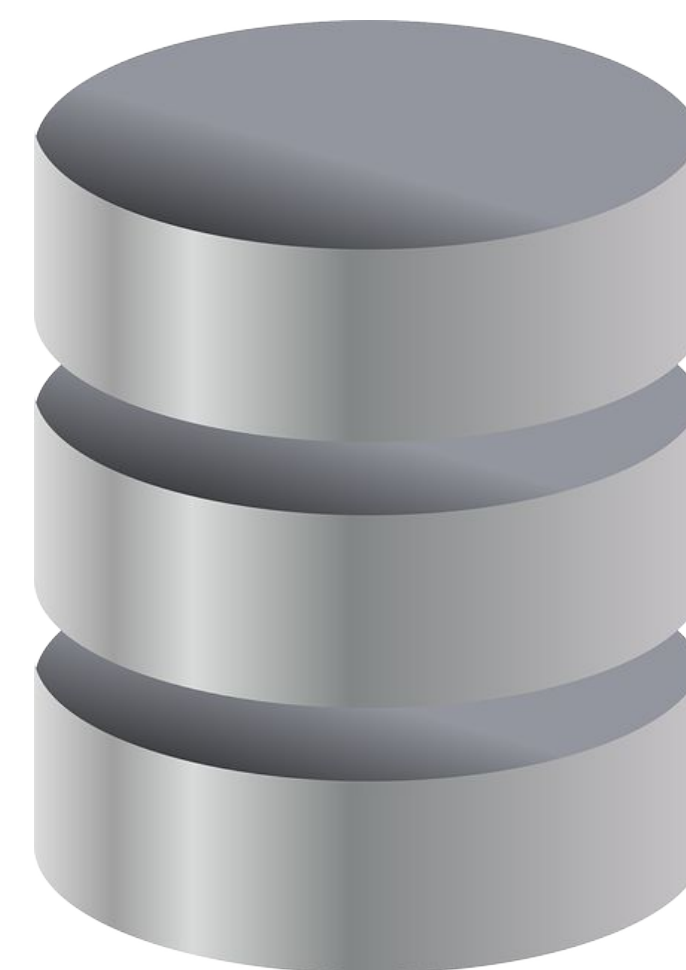
Database



Client



Server



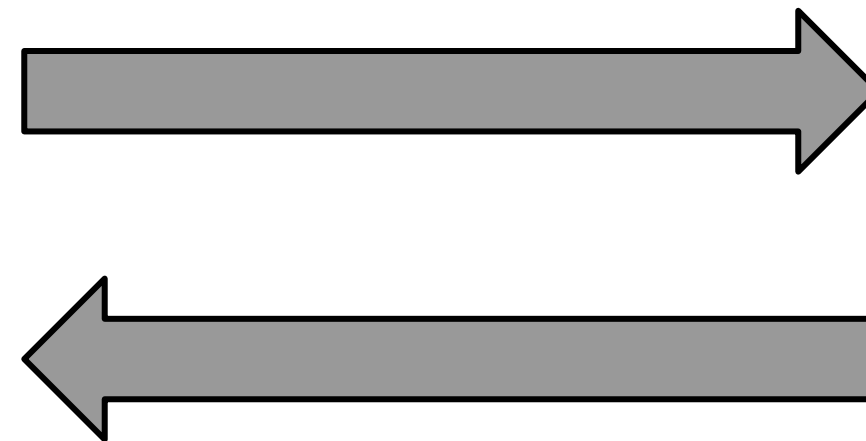
Database

Front End

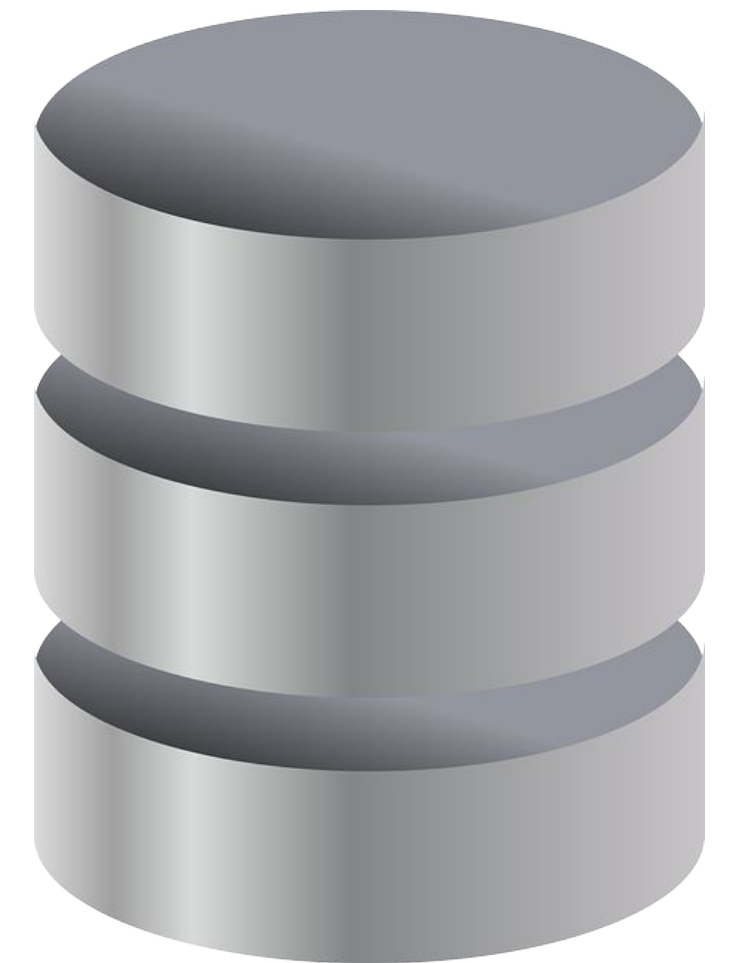
Back End



Client



Server



Database

Front End

Back End

HTML/CSS JavaScript

PHP

Ruby

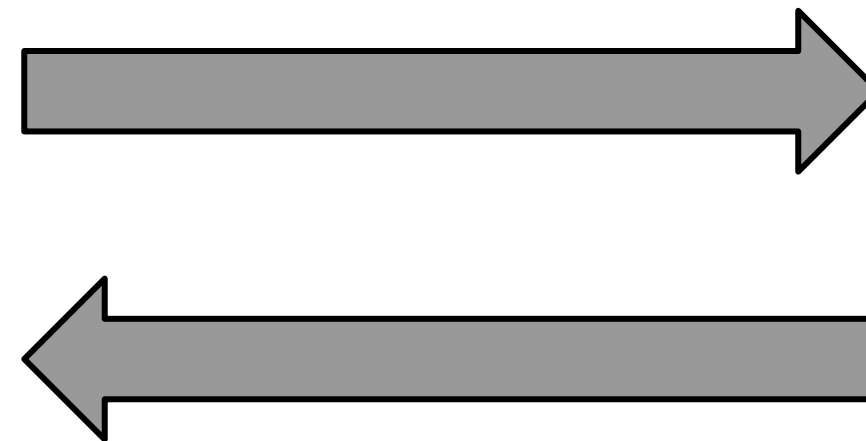
Node.js

MongoDB

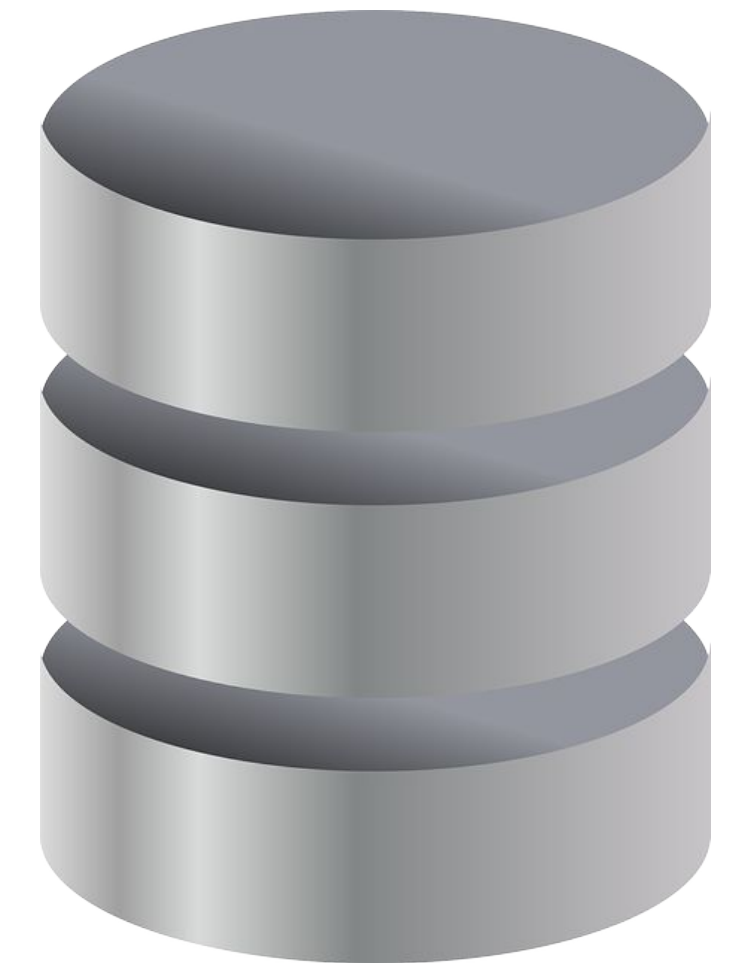
MySQL



Client



Server



Database

Application

Examples



Your App: Getting Started

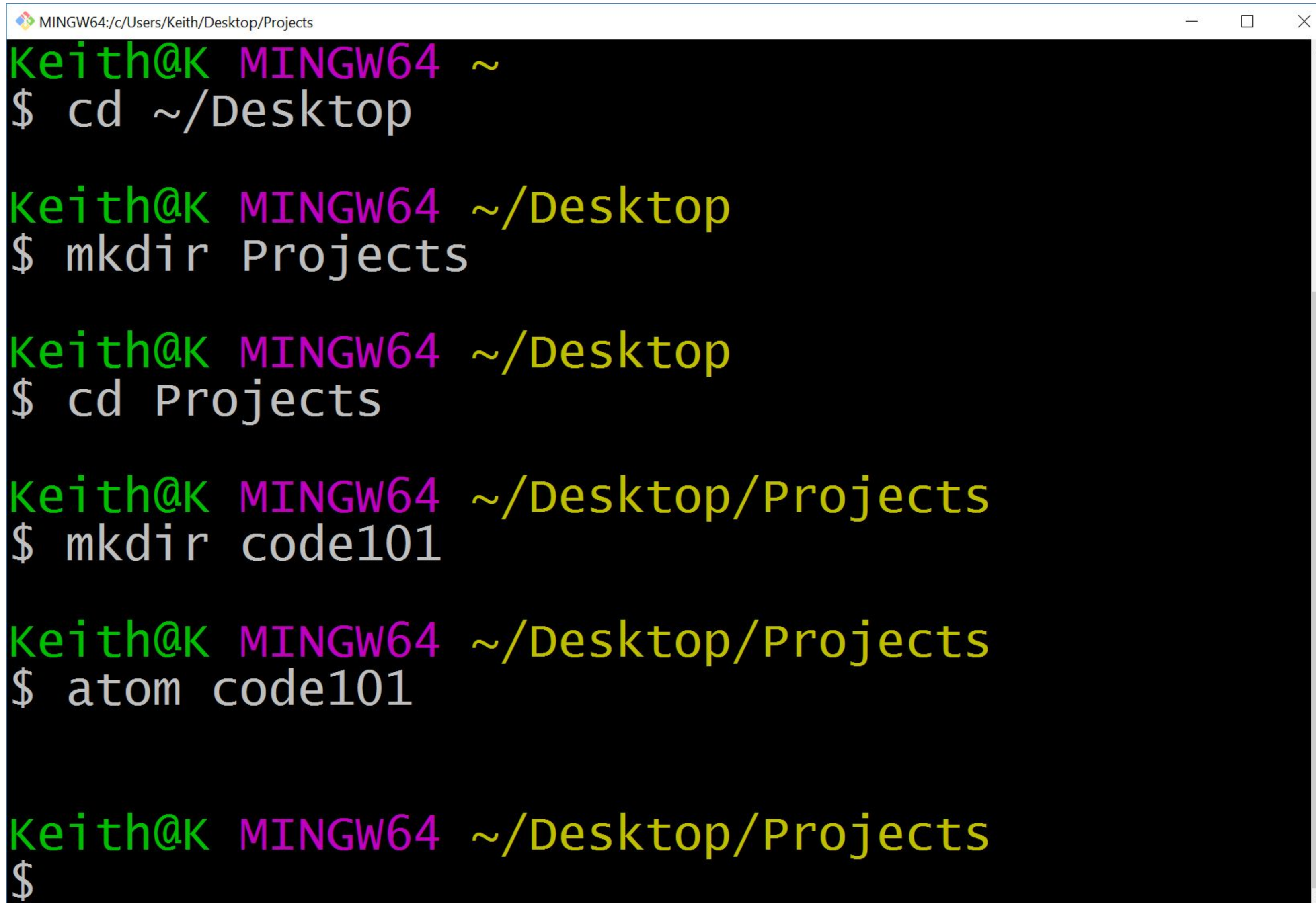
How To Start

- Decide on a cause (fictitious, or fictitious name). Identify your content. What do you need to communicate?
- **Pitch: your topic, need it addresses**
- Outline your content. Where do things go?
 - Pages needed, Headlines, Images, Paragraphs, Links
- Start sketching - organize into a template
- Make a Slack direct message group for your team
- Eventually, write code



Development Environment

Command Line (Mac: Terminal; Windows: Git Bash)



```
MINGW64:/c/Users/Keith/Desktop/Projects
Keith@K MINGW64 ~
$ cd ~/Desktop

Keith@K MINGW64 ~/Desktop
$ mkdir Projects

Keith@K MINGW64 ~/Desktop
$ cd Projects

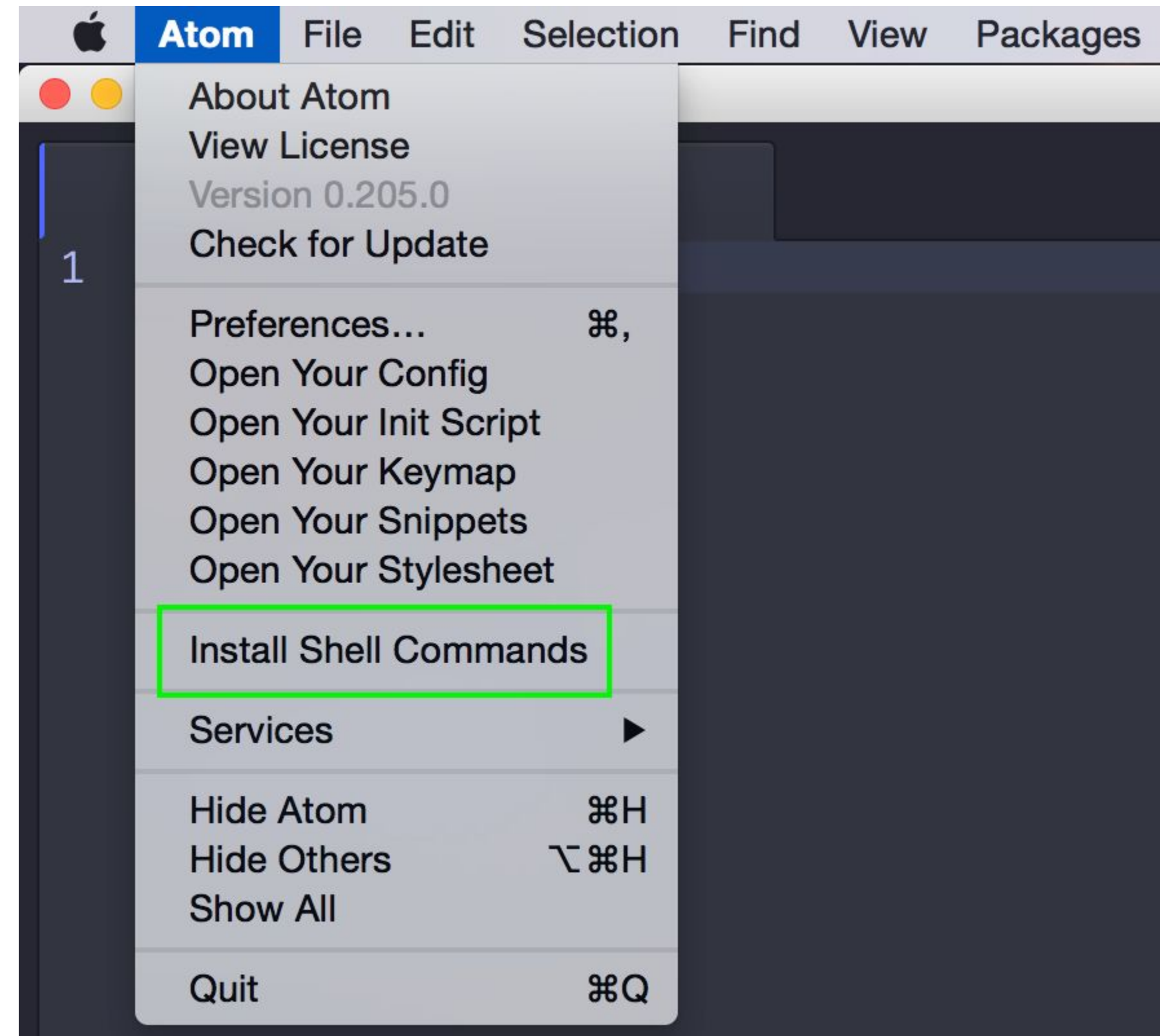
Keith@K MINGW64 ~/Desktop/Projects
$ mkdir code101

Keith@K MINGW64 ~/Desktop/Projects
$ atom code101

Keith@K MINGW64 ~/Desktop/Projects
$
```


Install Shell Commands

- `atom code101` didn't work on Mac?



ABOUT GIT

WITHOUT VERSION CONTROL:

Look familiar?

- `term_paper.docx`
- `term_paper2.docx`
- `term_paper2_with_footnotes.docx`
- `final_term_paper.docx`
- `term_paper_for_submission.docx`
- `term_paper_for_submission_for_real.docx`

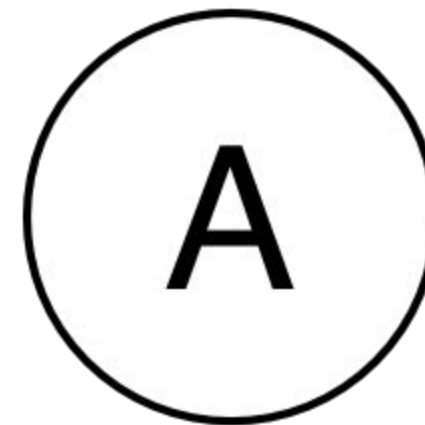
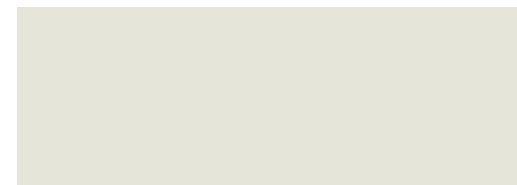
WHAT IS GIT?

It's a **version control system**.

- It lets multiple developers work on the same code
- A history of changes to your files
- The ability to view, apply, and remove those changes
- Keep all your project files in one **repository**
- It makes collaboration possible!

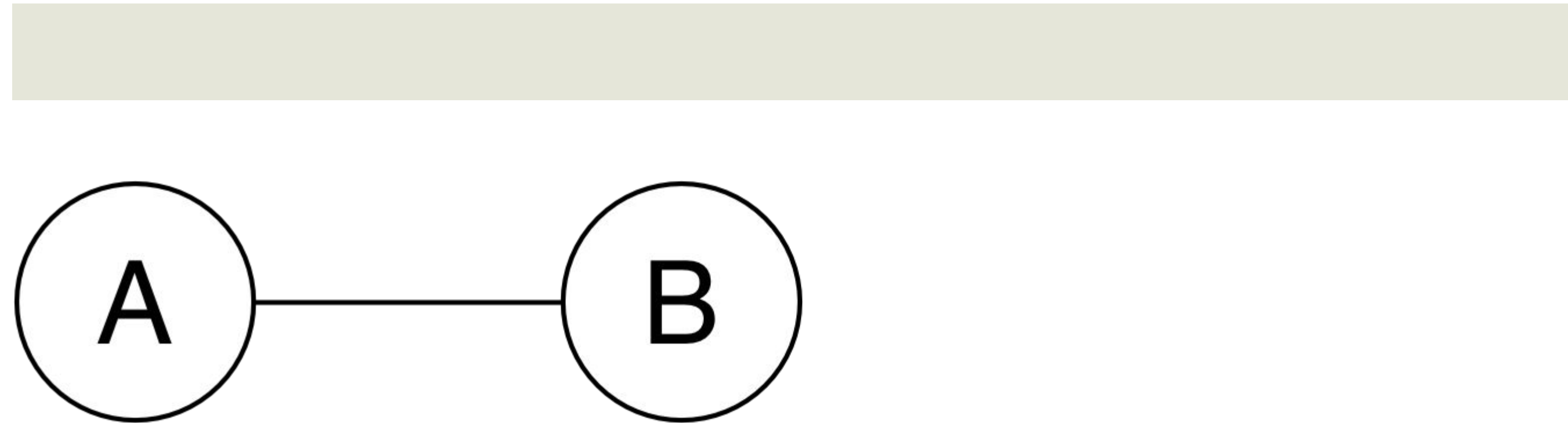
SNAPSHOTS IN TIME

- **Commits** represent each successive version of a file or files.
- Commits are the Git equivalent of "Save As..."



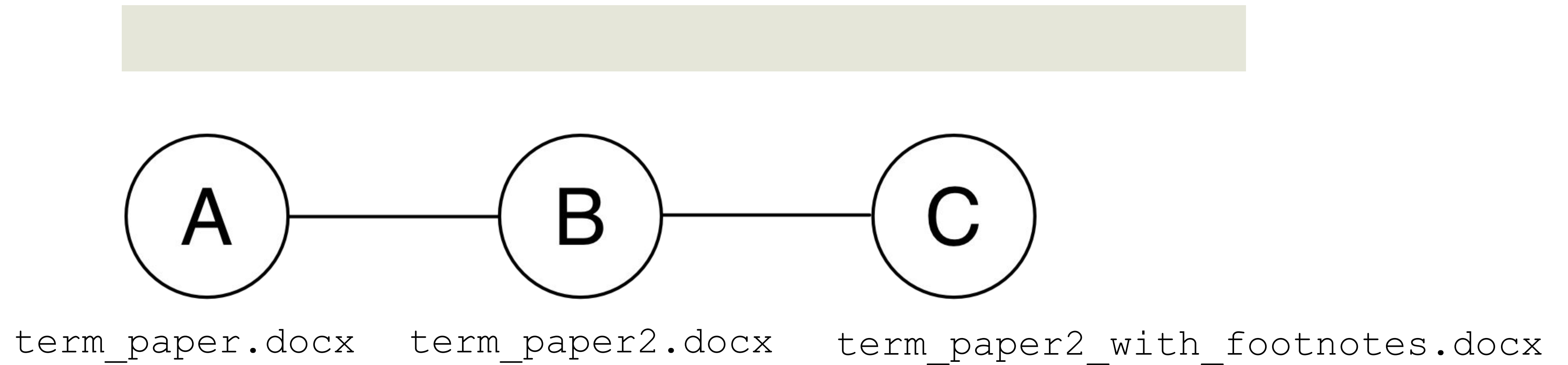
SNAPSHOTS IN TIME

Each successive version creates a new snapshot on the timeline of the project.

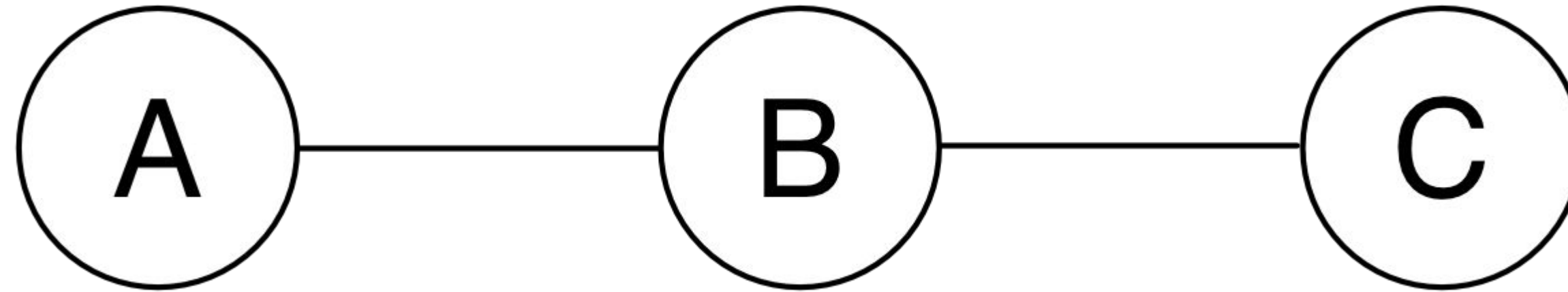


SNAPSHOTS IN TIME

Git keeps track of what the file looked like at different points in time.



A SUMMARY OF GIT



- You use Git to take snapshots of your code at points in time.
- Git keeps a history of what those snapshots look like.
- Every snapshot saves:
 - Message
 - Author
 - Date

GitHub

WHAT'S GITHUB?

- A way to share code with others!
- An online place to store your code. (Backup is good!)
- It uses Git to help you manage your team's work:
 - Version tracking
 - Reviewing changes
 - Keep changes separate until you want to add them in

GIT + GITHUB = AWESOME

With Git (version control) and GitHub (online code storage), you can:

- Have lots of team members work on the same files, without messing each other up
- Keep a history of each file over time
- Work on code on your own computer, and sync it with what's online

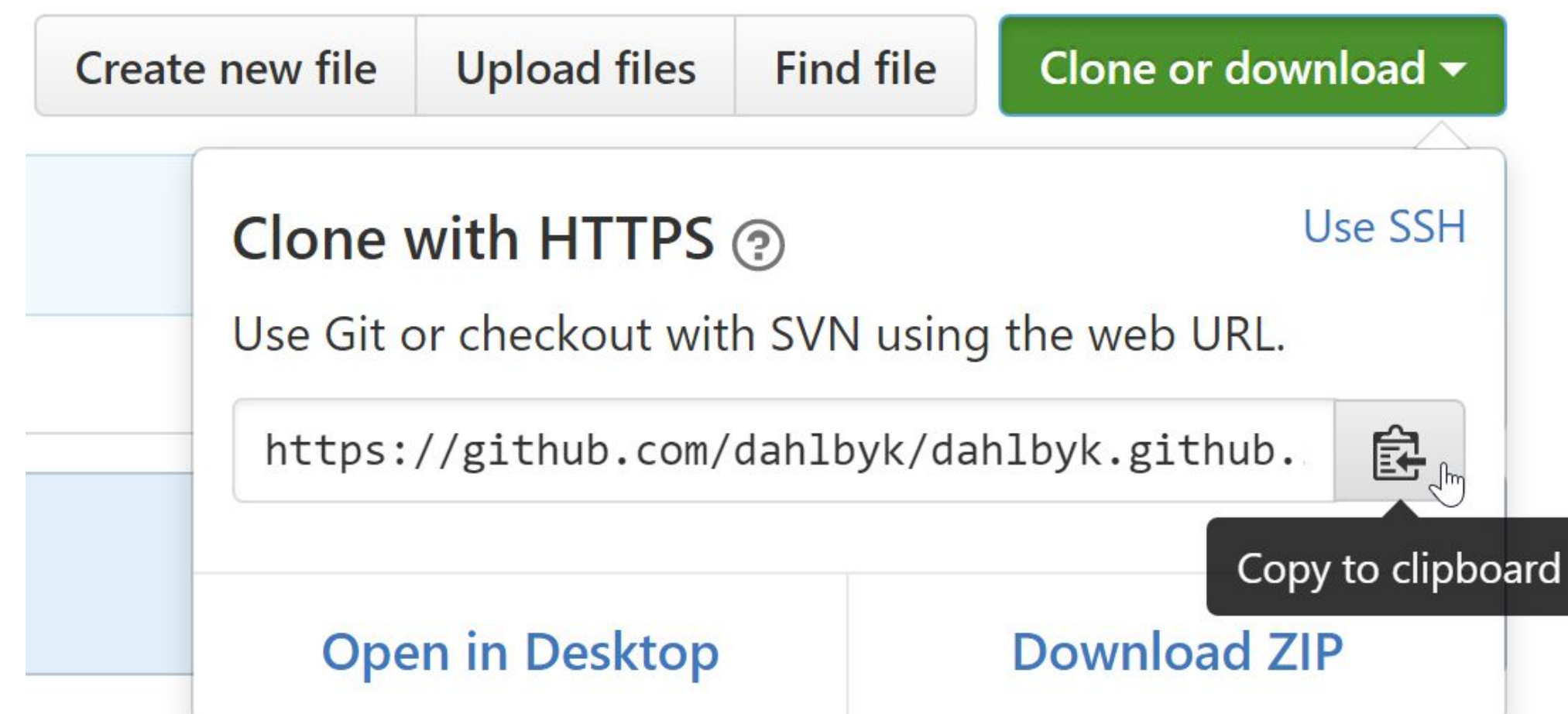
What's A Repository?

A repository is a collection of files.

- Usually, one project = one repository.
- Really large projects might have multiple repositories for different parts of their system (e.g. front end and back end).
- Repositories can live on GitHub or your computer.
- Now we need to make a copy of this repo on our computers.
- Repos can be connected, allowing them to send code to and receive code from each other.
- We'll do this by **cloning**.

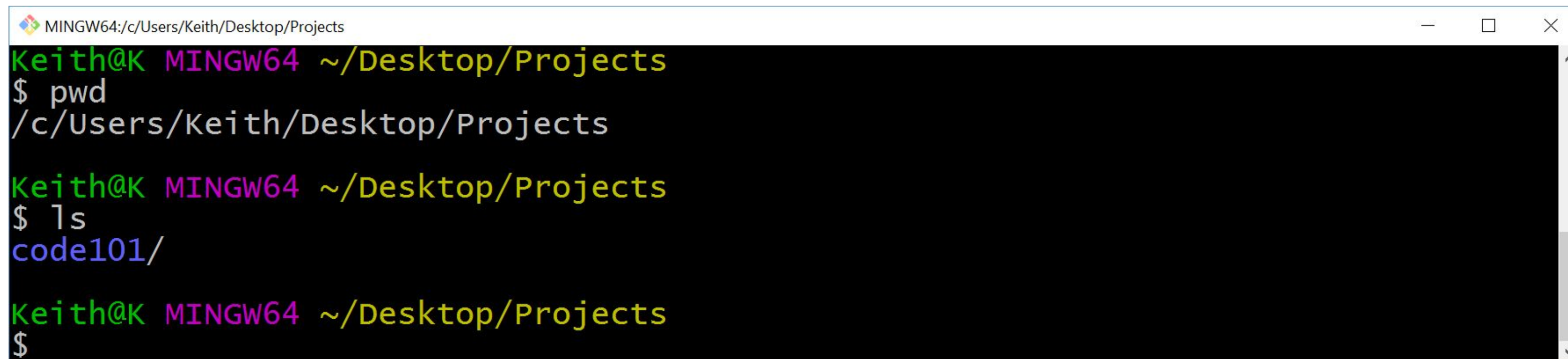
Clone That Repo

- Log in to GitHub and go to <https://github.com/DeltaVCode/code-101-templates>
- Click **Clone or download** dropdown
- Ensure it says **Clone with HTTPS**
- Copy the URL with *Copy to clipboard* or your keyboard shortcut



Go To Your Projects Folder

- Open or switch back to command line (Terminal or Git Bash)
- Type **pwd**
 - **P**rint **W**orking **D**irectory
 - It should show something like **.../Desktop/Projects**
 - If not, type **cd ~/Desktop/Projects**
- Type **ls** to list directory contents



```
MINGW64:/c/Users/Keith/Desktop/Projects
Keith@K MINGW64 ~/Desktop/Projects
$ pwd
/c/Users/Keith/Desktop/Projects

Keith@K MINGW64 ~/Desktop/Projects
$ ls
code101/

Keith@K MINGW64 ~/Desktop/Projects
$
```

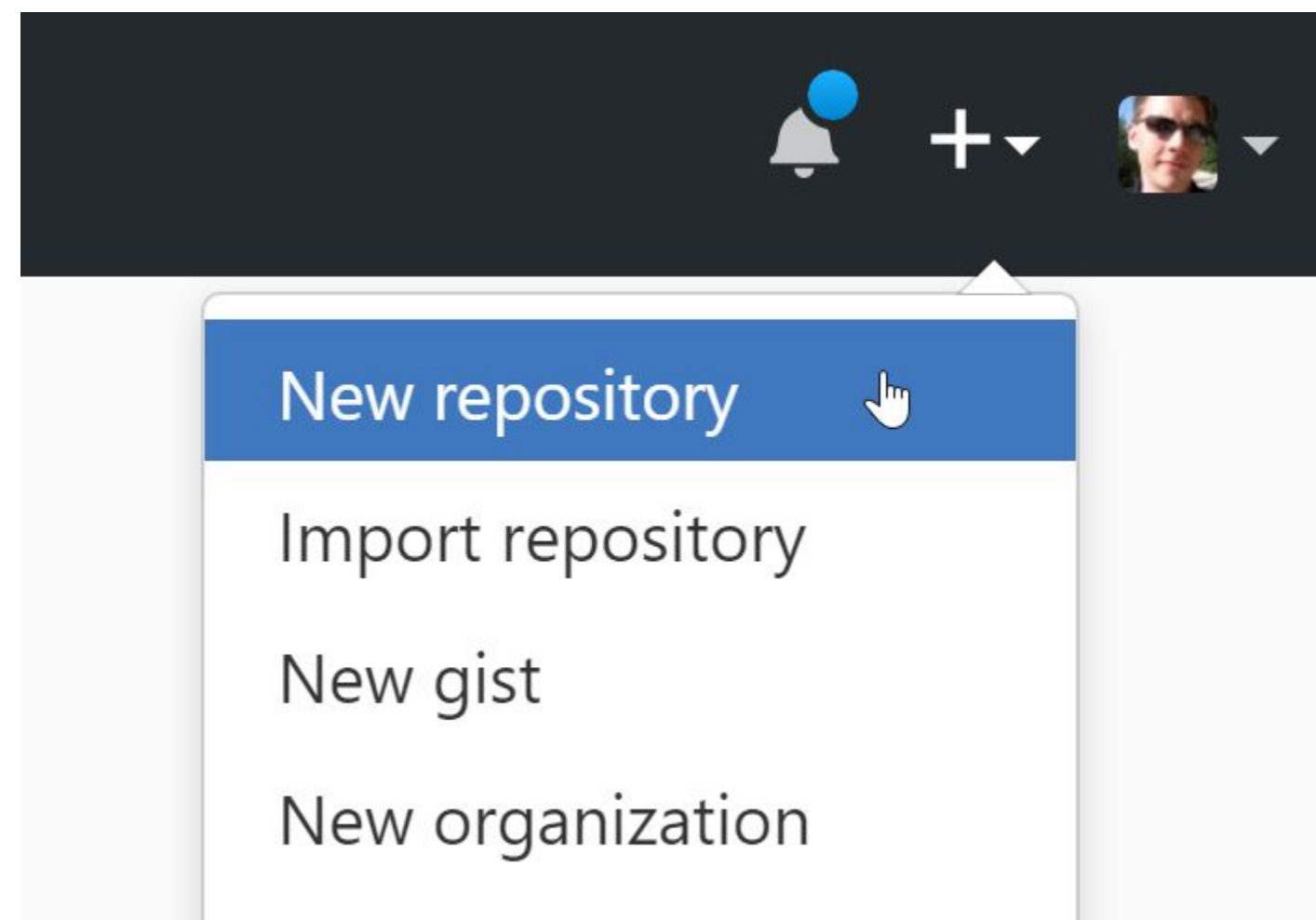
Git Command: **git clone**

In your Projects directory:

- Type **git clone** then paste in the link you just copied
 - Mac: command + V
 - Windows: right-click on the terminal window, select Paste
 - Linux: ctrl + shift + V
- Should look like this:
git clone https://github.com/*username*/...
- Hit **Enter**
- Now open the project in Atom:
 - **atom code-101-templates**

Creating A Repo On GitHub

- Now let's create our own repository!
- At the top right side of the Github window, look for your name and avatar.
- Next to it you'll find a small + sign. Click that.
- From the menu that opens, select *New repository*.



Creating A Repo On GitHub

- Repository name: **username.github.io**
- Check ***Initialize this repository with a README.***
- Click ***Create repository.***

Owner

 dahlbyk ▾

/

Repository name

dahlbyk.github.io ✓

Great repository names are short and memorable. Need inspiration? How about [fuzzy-funicular](#).

Description (optional)

☒  **Public**

Anyone can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

☒ **Initialize this repository with a README**

This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: None ▾

Add a license: None ▾



Create repository

GitHub Pages

- Clone this repository just like the previous one.
 - Be sure to clone inside of projects, not code-101-templates
- Congrats! You just made your first repo.
- From the Github repository, click Settings, find GitHub Pages and change the source to master branch.
- Your web page will be live!

`https://username.github.io/`

- Learn more at <https://pages.github.com/>

Git Command: **git remote**

What just happened?

- You made a local repo that is a copy of the one on GitHub.
 - **cd *username.github.io***
- You told that local repo that it can talk to the GitHub one.
 - **git remote -v**
- It made a directory that has all the files in it you had online.
- Now open new project in Atom:
 - **atom .**

Git Cheat Sheet

Navigating the Terminal

cd folder-name

Type **cd** folder-name to navigate to a folder.¹ Type **cd ..** to go to the parent folder.

ls -la

Display ALL (including hidden) files in the folder. **-la** is a flag. Flags modify commands.²

Using Git

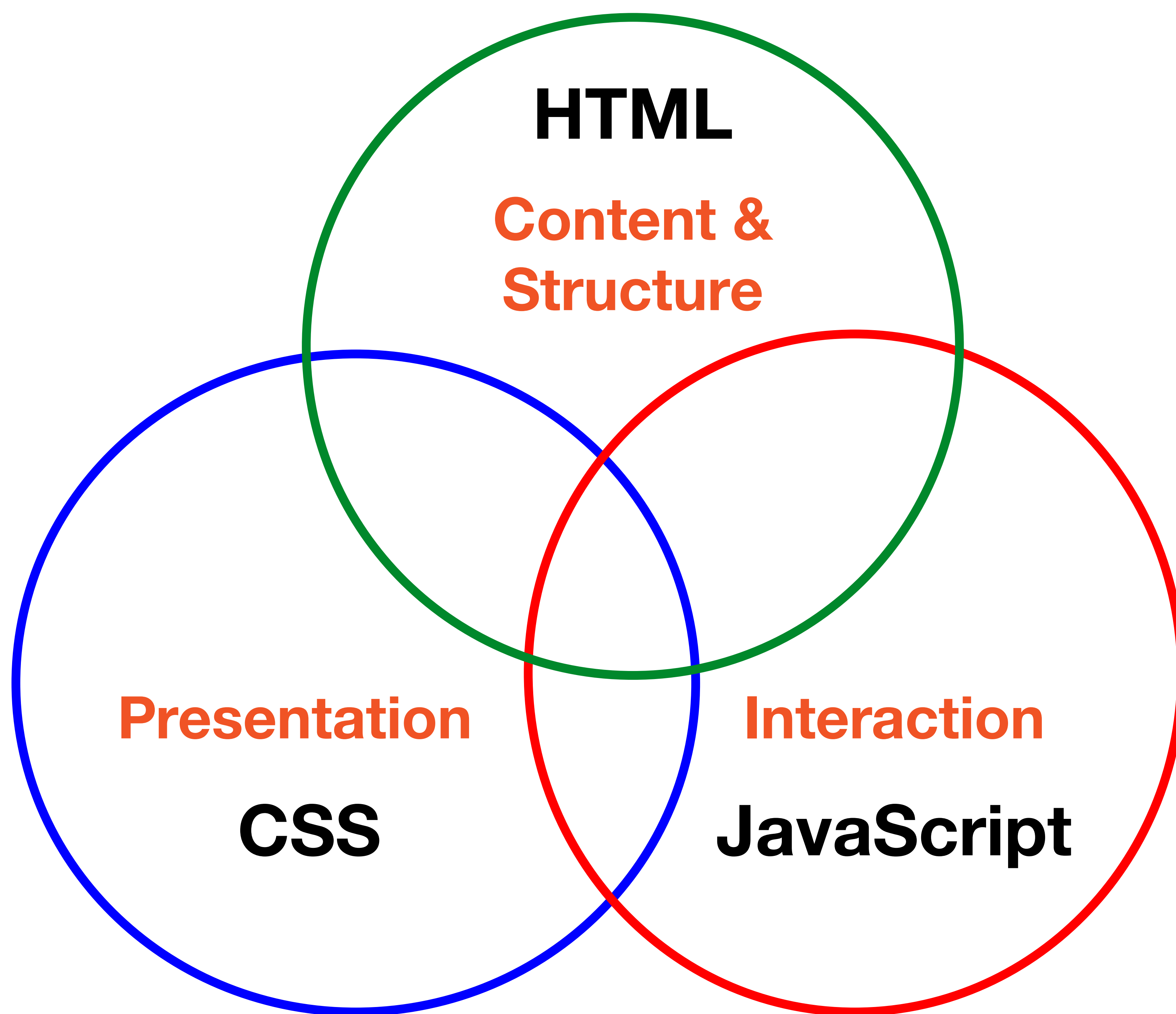
git clone url

Replace url with the url of a repo you want to clone to your computer.

¹ Start typing the folder name and hit tab to auto-complete.

² Flag letters are abbreviations. **l** stands for list and **a** stands for all. You can mix and match these freely. Try **ls**, **ls -a**, and **ls -l** to see the differences.

The Front End





HTML

HTML

- HTML = Hypertext Markup Language
- Tells the browser **what** the content is
 - Text
 - Headlines
 - Images
 - Links
- Communicates how that content is **organized**
 - Importance
 - Blocks

Demo

Earthquake Preparedness

- [North America](#)
- [East Asia](#)
- [Southeast Asia](#)



North America

Look around places where you spend time. Identify safe place such as under a sturdy piece of furniture or against an interior wall in your home, office or school so that when the shaking starts, you Drop to the ground. Cover your head and neck with your arms, and if a safer place is

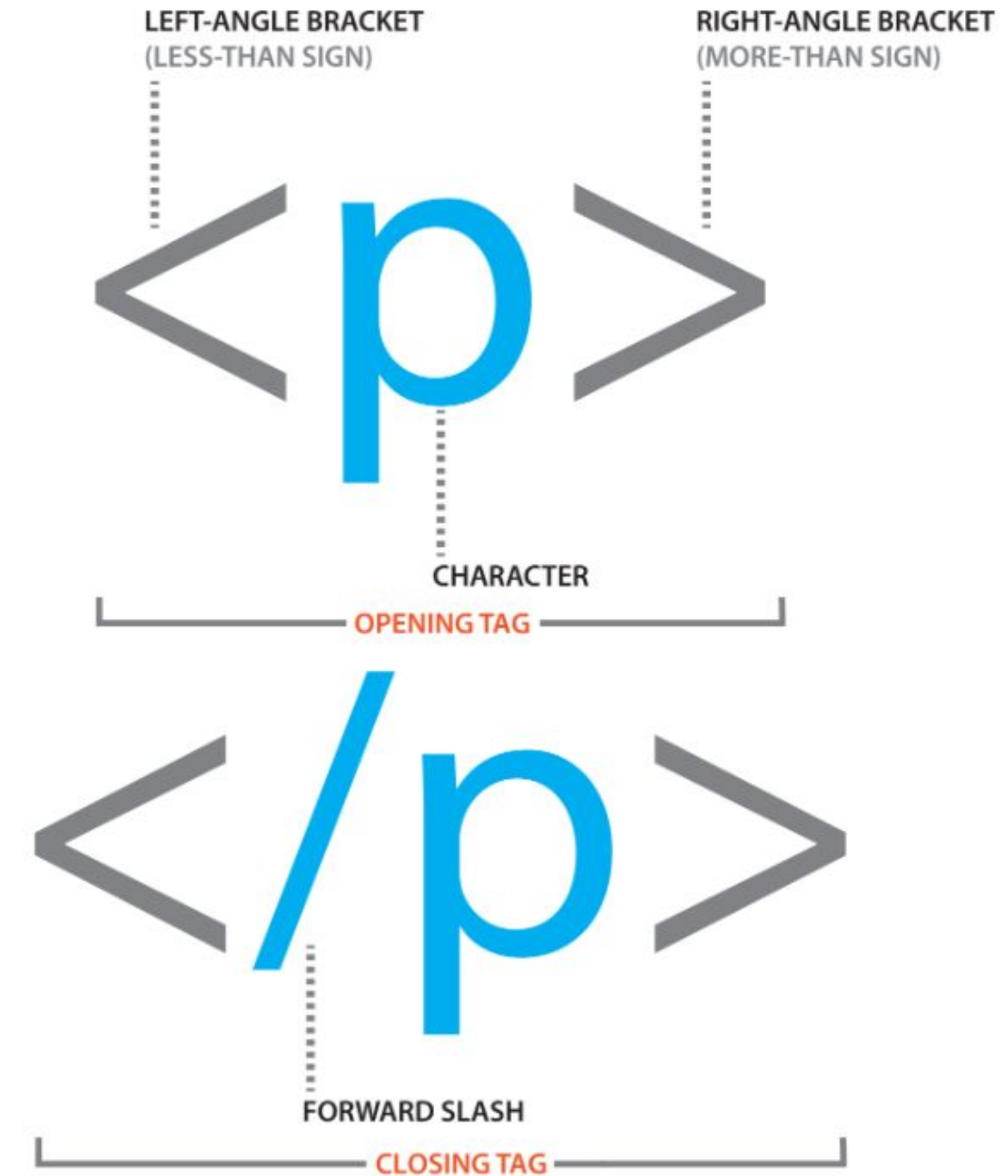
How HTML Works

- Content is wrapped in "**tags**"
- Tags tell the browser the **meaning** of the content
- Browsers **display** the content according to the rules of the tags
- This is called "**marking up**" the content
- Ergo: Hyper Text Markup Language

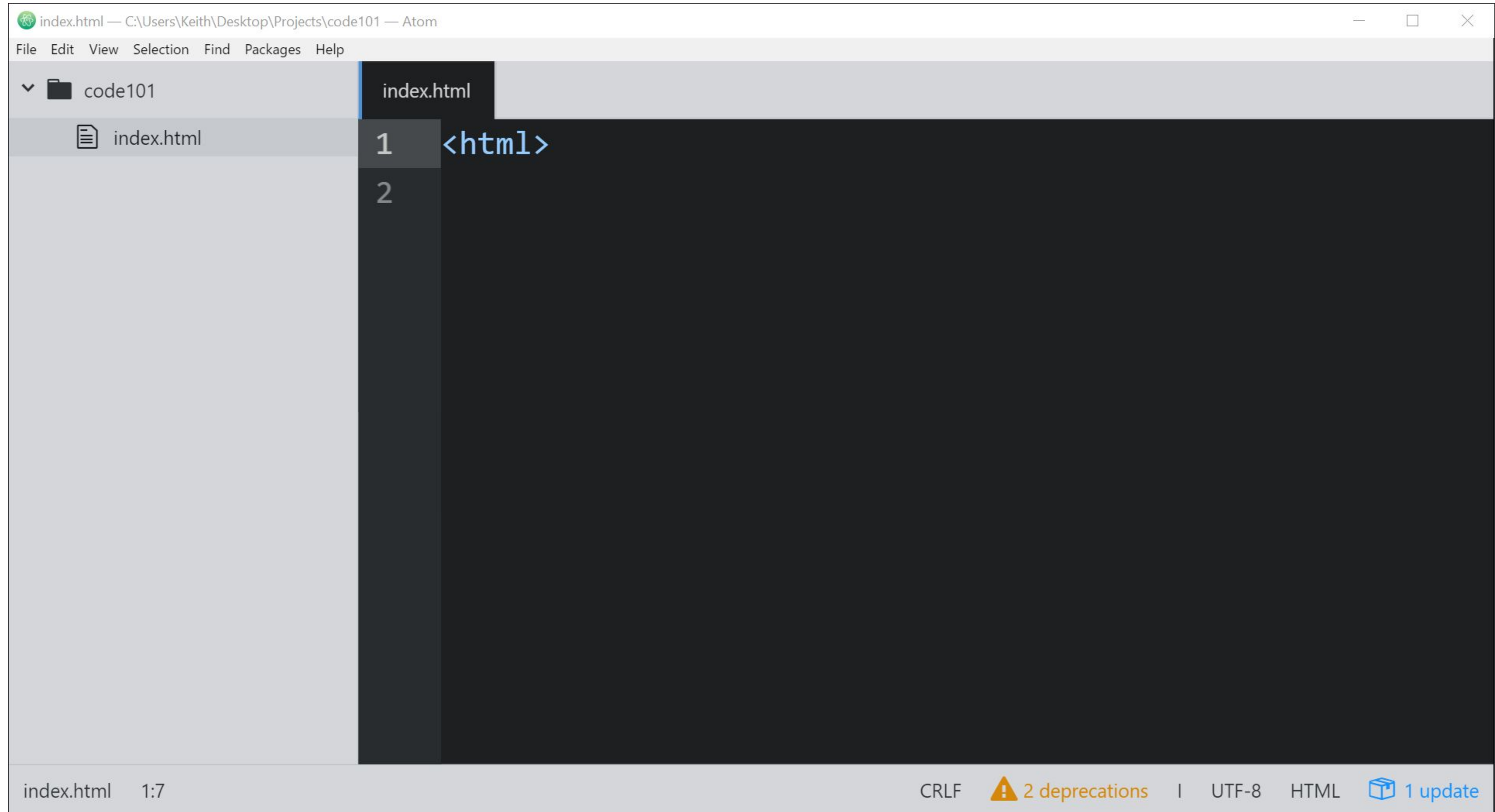
HTML

- HTML Tags

- Tags surround the content they describe. Like this:
- There are beginning tags: `<p>`
And ending tags: `</p>`
- Browsers don't care whether you use upper case or lower case.
- Content is placed between tags.



First HTML File



Anatomy Of A Web Page

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>Welcome to DeltaV</title>
```

```
  </head>
```

```
  <body>
```

```
    <!-- content of web page goes here -->
```

```
  </body>
```

```
</html>
```


How HTML Handles White Space

- Whitespace = tabs, spaces, returns
- All white space characters show up as one space.
- Multiple whitespace characters show up as one space.
- We can use different tags to make:
 - Double-spaced lines: `<p>My paragraph</p>`
 - Single line break: `<br>`
 - Lists: `<ul></ul>` and `<li></li>`

Organize Your Content

- HTML has tags that are applied to blocks of content.
 - `<header></header>`
 - `<nav></nav>`
 - `<section></section>`
 - `<article></article>`
 - `<aside></aside>`
 - `<footer></footer>`

HTML

- HTML Headings

- Tells the browser how important a line of text is

`<h1>I'm the biggest heading!</h1>`

`<h6>And I'm the smallest heading.</h6>`

- They are automatically shown in bold, in a bigger size, and with a line break after them.

Attributes

- Tags can have **attributes** that give more information:
 - Where to find an image file
 - The URL of a link
- Attributes have different names, but the syntax is the same:

```

```

```
<a href="about_us.html">About Us</a>
```
- For local HTML & image file names: lowercase, no spaces

HTML - What It Looks Like

```
<body>
```

```
  <header>
```

```
    <h1>Earthquake Preparedness</h1>
```

```
  </header>
```

```
  <nav>
```

```
    <ul>
```

```
      <li><a href="">North America</a></li>
```

```
      <li><a href="">East Asia</a></li>
```

```
      <li><a href="">Southeast Asia</a></li>
```

```
    </ul>
```

```
  </nav>
```

HTML - Structure

```
<body>
```

```
  <header>
```

```
    <h1>Earthquake Preparedness</h1>
```

```
  </header>
```

```
  <nav>
```

```
    <ul>
```

```
      <li><a href="">North America</a></li>
```

```
      <li><a href="">East Asia</a></li>
```

```
      <li><a href="">Southeast Asia</a></li>
```

```
    </ul>
```

```
  </nav>
```


HTML - Meaning

```
<body>
```

```
  <header>
```

```
    <h1>Earthquake Preparedness</h1>
```

```
  </header>
```

```
  <nav>
```

```
    <ul>
```

```
      <li><a href="">North America</a></li>
```

```
      <li><a href="">East Asia</a></li>
```

```
      <li><a href="">Southeast Asia</a></li>
```

```
    </ul>
```

```
  </nav>
```

HTML - Content

```
<body>
```

```
  <header>
```

```
    <h1>Earthquake Preparedness</h1>
```

```
  </header>
```

```
  <nav>
```

```
    <ul>
```

```
      <li><a href="">North America</a></li>
```

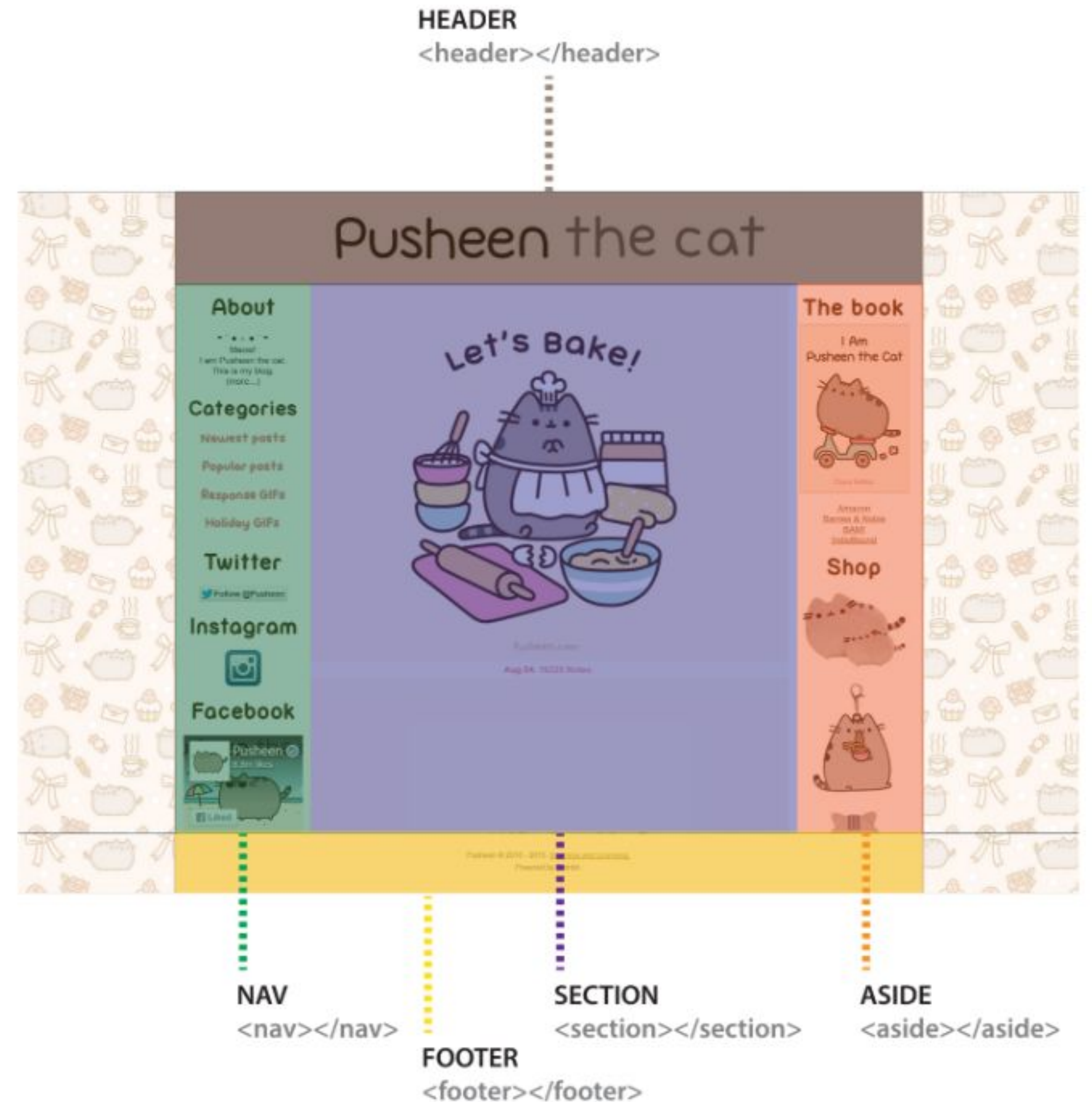
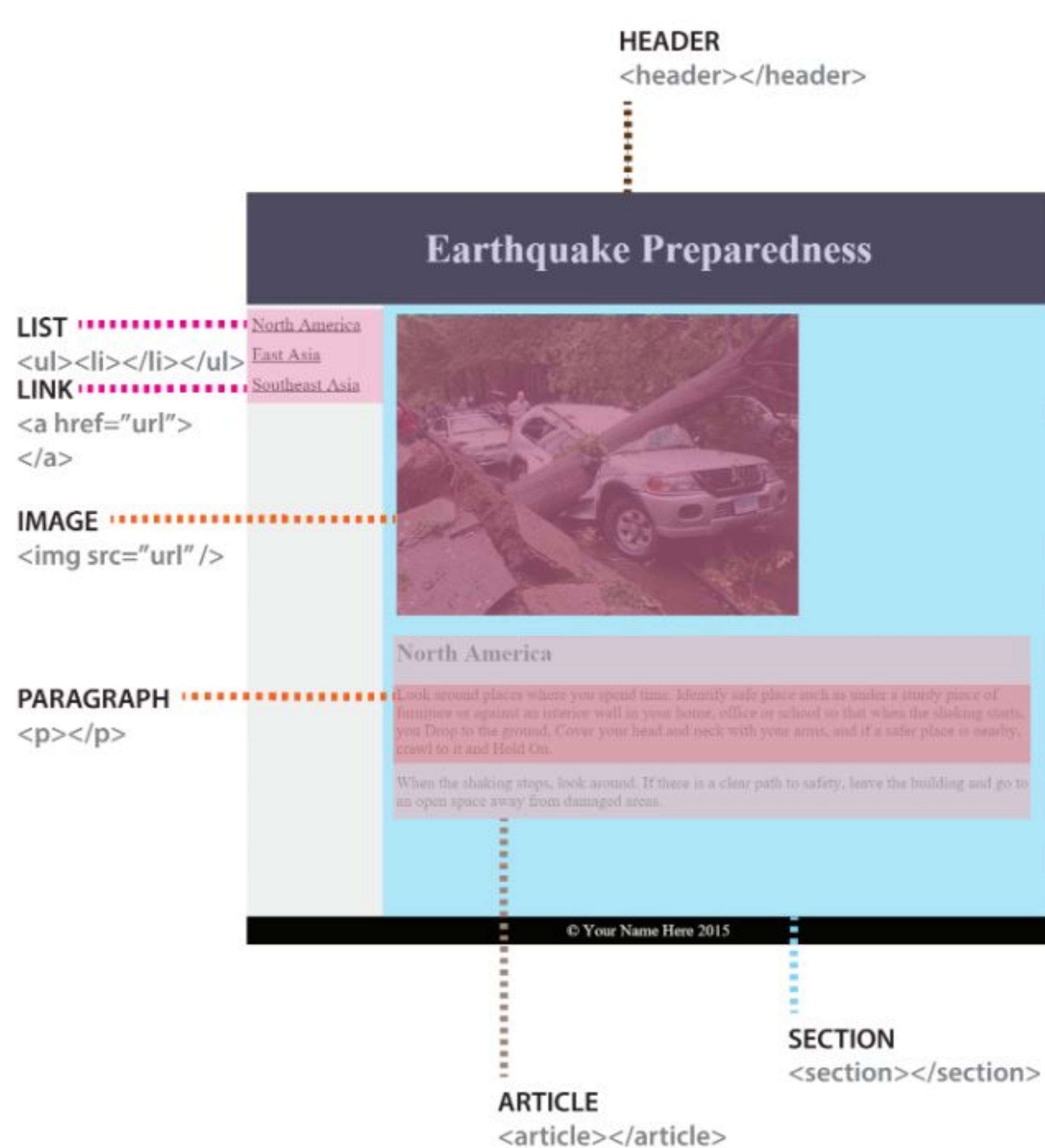
```
      <li><a href="">East Asia</a></li>
```

```
      <li><a href="">Southeast Asia</a></li>
```

```
    </ul>
```

```
  </nav>
```


HTML - Site Anatomy



HTML - Cheat Sheet

<head></head>

This tag contains content ABOUT the page like the **title**.

<title></title>

This tag contains the title that shows up in the browser title bar.

<body></body>

This tag contains content shown ON the page like a **paragraph**.

<p></p>

Surround each paragraph of content with this tag.

Create links! The content between the tags is the display text.

This tag inserts an image. It doesn't need to be closed.

This tag **bolds** the content between it.

This tag creates a line break. It does not require a closing tag.

<h1></h1>

Use this tag with a number 1-6 to define a heading. <h1> is the largest and <h6> the smallest.

Troubleshooting



SHARING CODE

SHARE IT!

- **status, add, commit & push** to share on GitHub
- See your code live on the web!

<https://username.github.io/>

COMMIT AND PUSH

GIT COMMAND: `git status`

Now that your files are in your repo, we need to make a commit (take a snapshot of them).

- Review the current status of your files by typing **git status**

```
$ git status
```

- It will tell you what files have changed since your last commit.
- In this case, you've only made one commit - when you told it to put a README in your repo (online).

GIT COMMAND: `git add`

Next: we need to tell git what files to commit.

- This is done by typing **git add** and then a filename.

```
$ git add index.html
```

- It will tell you what files have changed since your last commit.
- Repeat this command for each file that you added.
- This tells the file to get ready for the snapshot.
- Type **git status** again to see the difference!

GIT COMMAND: `git commit`

Finally, take that snapshot!

- Type **`git commit -m "your message goes here"`**

```
$ git commit -m "First commit"
```

- **`git commit`** is what takes the snapshot
- **`-m`** tells it that you want to create a message with the commit
- Think of the messages as being like photo captions.

GIT COMMAND: `git push`

Great! Now it's time to copy this code to your repo on GitHub.

- Type **git push origin master**

```
$ git push origin master
Counting objects: 6, done.
...
To https://github.com/dahlbyk/dahlbyk.github.io
* [new branch]      master -> master
```

- This sends this commit (this snapshot of your code) to GitHub.
- Go to your repo on GitHub, and look for your files!

VERIFY ON GITHUB

In your browser on GitHub you will see the file(s) that you pushed as well as the commit message.

2 commits

1 branch

0 releases

1 contributor

Branch: master

New pull request

Create new file

Upload files

Find file

Clone or download

 dahlbyk committed on GitHub Create README.md

Latest commit bde85b7 just now

README.md

Create README.md

just now

index.html

Links

6 years ago

README.md

dahlbyk.github.io

SHOW OFF!

Congratulations! You have successfully deployed your awesome work on the internet.

Now you can share that link with friends and family, and they can see what you did.

Give yourself a pat on the back!

Git Cheat Sheet

`git add --all`

This command will stage ALL untracked files.

`git status`

It shows which updated files are untracked (not git added) and which are staged.

`git commit -m "msg"`

Commit ALL staged files. Replace `msg` with a descriptive message.

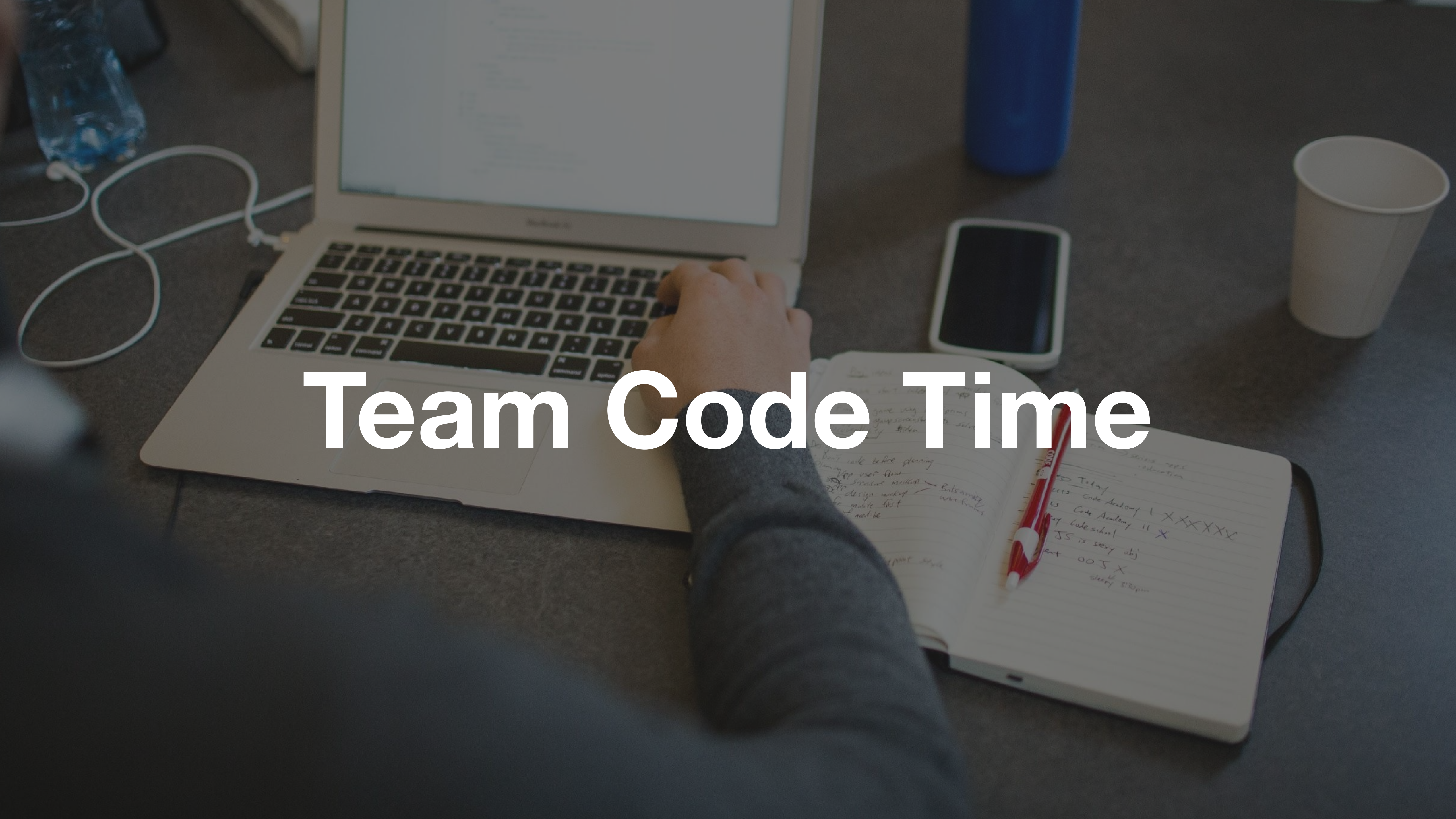
`git push origin master`

This command will upload your commit to GitHub.

Create new files

- Save index.html with the file names you used inside the tags.
- Delete the main content out of each, leaving the basic HTML structure and headings.
- Test the links.
- Do an ACP

Team Code Time



Write Your HTML

- Assign one "home" page, called `index.html`
- Decide other page filenames (`about_us.html`, etc)
 - **Reminder: lowercase, no spaces!**
- Decide who is going to build which page(s)
- Create your page(s) in Atom
 - Add `<html>`, `<head>`, `<title>` and `<body>` tags
 - Between the `<body>` tags, write your content
 - Then, add sections, headers, nav, paragraphs, etc
 - Add navigation to team's pages
 - Use team's wireframe as a guide to create your HTML



CSS

Why



- CSS = Cascading Style Sheets
- Design and lay out your HTML elements. (The "look and feel")
- Maintain consistent styles.

CSS

- CSS works by:
 - **Selecting** elements to apply a **style** to
 - Defining what that **style** is: color, font, spacing, etc
- Tells the browser how the content should **look**
 - Fonts
 - Colors
 - Placement of content
 - Responsive: adapts to the user's screen size

Demo

Earthquake Preparedness

[North America](#)

[East Asia](#)

[Southeast Asia](#)



Insert your
advertisement
here.

North America

Look around places where you spend time. Identify safe place such as under a sturdy piece of furniture or against an interior wall in your home, office or school so that when the shaking starts, you Drop to the ground, Cover your head and neck with your arms, and if a safer place is nearby, crawl to it and Hold On.

Earthquake Preparedness

North

America

East Asia

Southeast

Asia



Insert your
advertisement
here.

North America

Look around places where you spend time. Identify safe place such as under a sturdy piece of furniture or against an interior wall in your home, office or school so that when the shaking starts, you Drop to the ground, Cover your head and neck with your arms, and if a safer place is nearby, crawl to it and Hold On.

When the shaking stops, look around. If there is a clear path to safety, leave the building and go to an open space away from damaged areas.

Including CSS In Your HTML

- CSS Styles can be written inline next to thing you want to style, in the header, or in an external file referenced in the header.

- **externally linked - in the HTML's header (preferred)**

```
<link href="styles.css" rel="stylesheet">
```

- **inline - an HTML tag in <head>**

```
<style> p { color: blue } </style>
```

- **inline - in HTML attributes**

```
<p style="color: blue">link</a>
```

CSS: Selecting By Tag

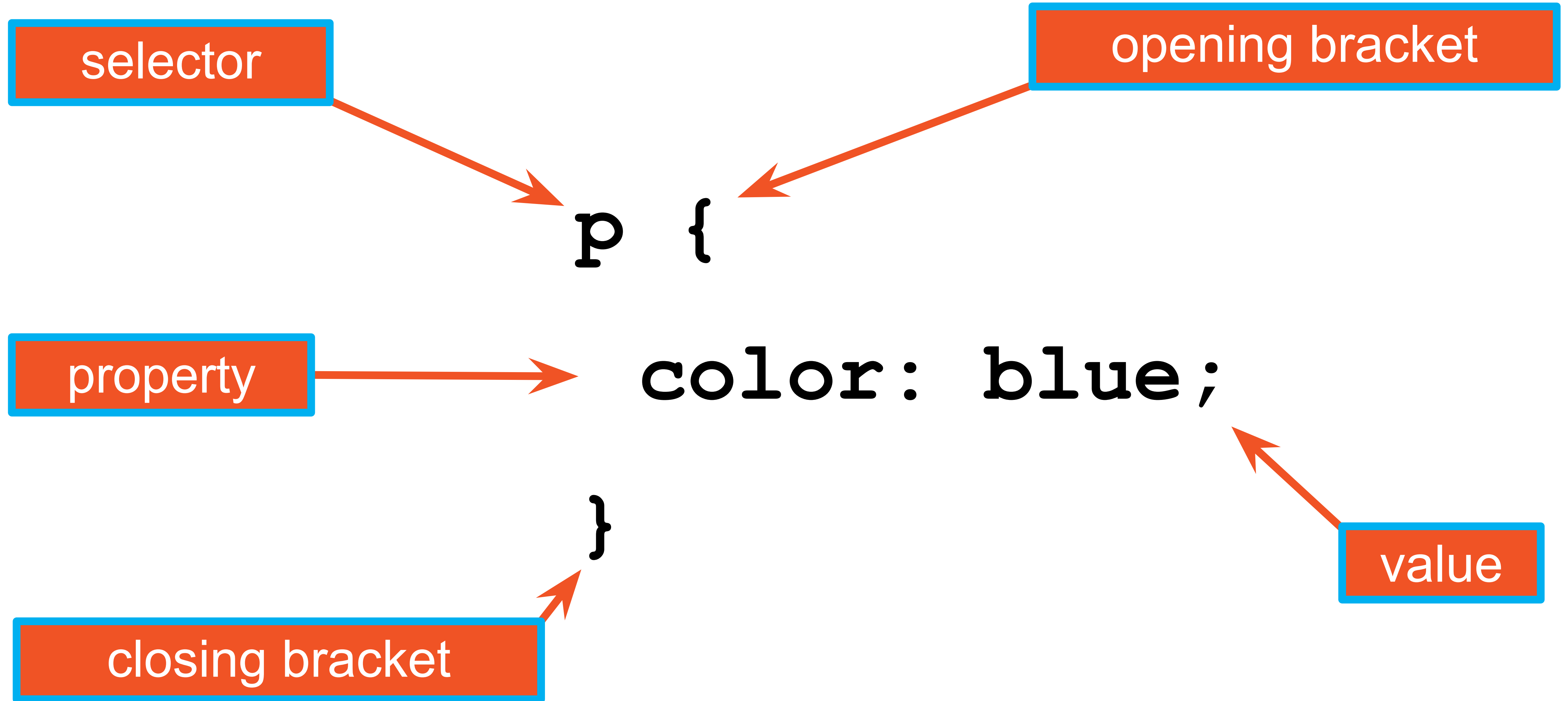
- In your HTML
 - Content areas you want to style

1. `<p></p>`
2. `<nav></nav>`
3. `<ul></ul>`
4. `<li><a></a></li>`

- In your CSS
 - Selecting content areas to style by **tag**

1. `p { }`
2. `nav { }`
3. `ul { }`
4. `li a { }`

CSS: What It Looks Like



CSS - What It Looks Like

```
h1 {  
    color: white;  
    background-color: black;  
    font-family:  times new roman;  
}
```

```
a {  
    color: black;  
}
```

```
img {  
    height: 400px;  
    width: 400px;  
}
```

Some CSS Properties

- `color: crimson;`
- `background-color: beige;`
- `font-size: 14px;`
- `font-weight: bold;`
- `font-family: Arial, Helvetica, sans-serif;`
- `font-style: italic;`
- `text-decoration: underline;`
- `line-height: 24px;`

px = pixels

CSS - Cheat Sheet

width	The content area's width (within the padding, border, and margin)	font-size	Specify the size of the font using a value or keyword.
margin	The margin of the element on each of its four sides.	text-align	Set the horizontal alignment of text using a keyword.
padding	The padding of the element on each of its four sides.	vertical-align	Set the vertical alignment of text using a keyword.
background-color	Set the background color of an element with a value or keyword.	display	The way an element is rendered on screen. This setting can affect the position of other elements.
color	Set the text color of an element with a value or keyword.		


```
header {  
  color: white;  
  background-color:  
  black;  
  text-align: center;  
  font-size: 20px;  
  display: block;  
}
```

```
nav {  
  background-color:  
  #eeeeee;  
  width: 125px;  
  height: 600px;  
  display: inline-block;  
  font-size: 18px;  
}
```

```
body {  
  width: 800px;  
}
```

Earthquake Preparedness

[North America](#)
[East Asia](#)
[Southeast Asia](#)



North America

Look around places where you spend time. Identify safe place such as under a sturdy piece of furniture or against an interior wall in your home, office or school so that when the shaking starts, you Drop to the ground, Cover your head and neck with your arms, and if a safer place is nearby, crawl to it and Hold On.

When the shaking stops, look around. If there is a clear path to safety, leave the building and go to an open space away from damaged areas.

© Your Name Here 2015

```
section {  
  width: 640px;  
  display: inline-block;  
  padding: 10px;  
}
```

```
footer {  
  width: 800px;  
  background-color:  
  black;  
  color: white;  
  display: block;  
  text-align: center;  
}
```

HTML: Generic Elements

- Generic **block** element

- `<div>...</div>`

- Generic **inline** element

- `<span>...</span>`

```
<div id="container">
```

```
  <p>A <span class="highlight">highlighted</span>
```

```
word.</p>
```

```
</div>
```

CSS: Selecting By class Or id

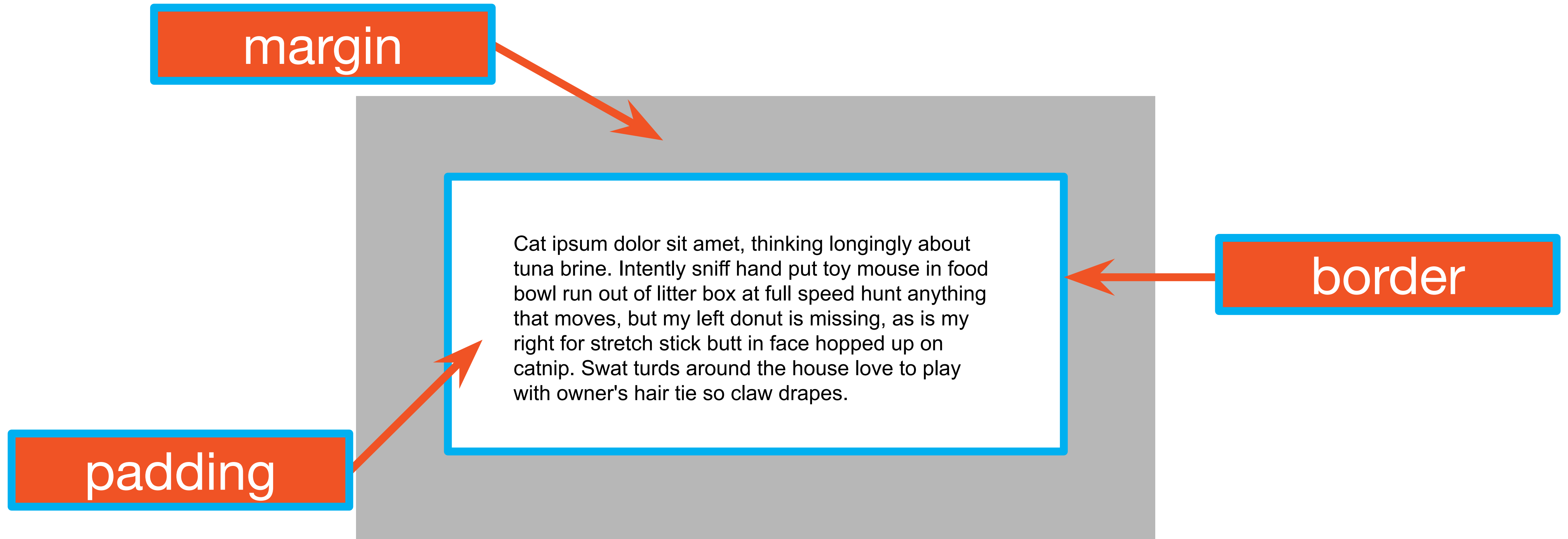
- **By class**

- Can be used multiple times within HTML.
- The HTML: `<span class="highlight">...</span>`
- The CSS selector: `.highlight { color: red }`

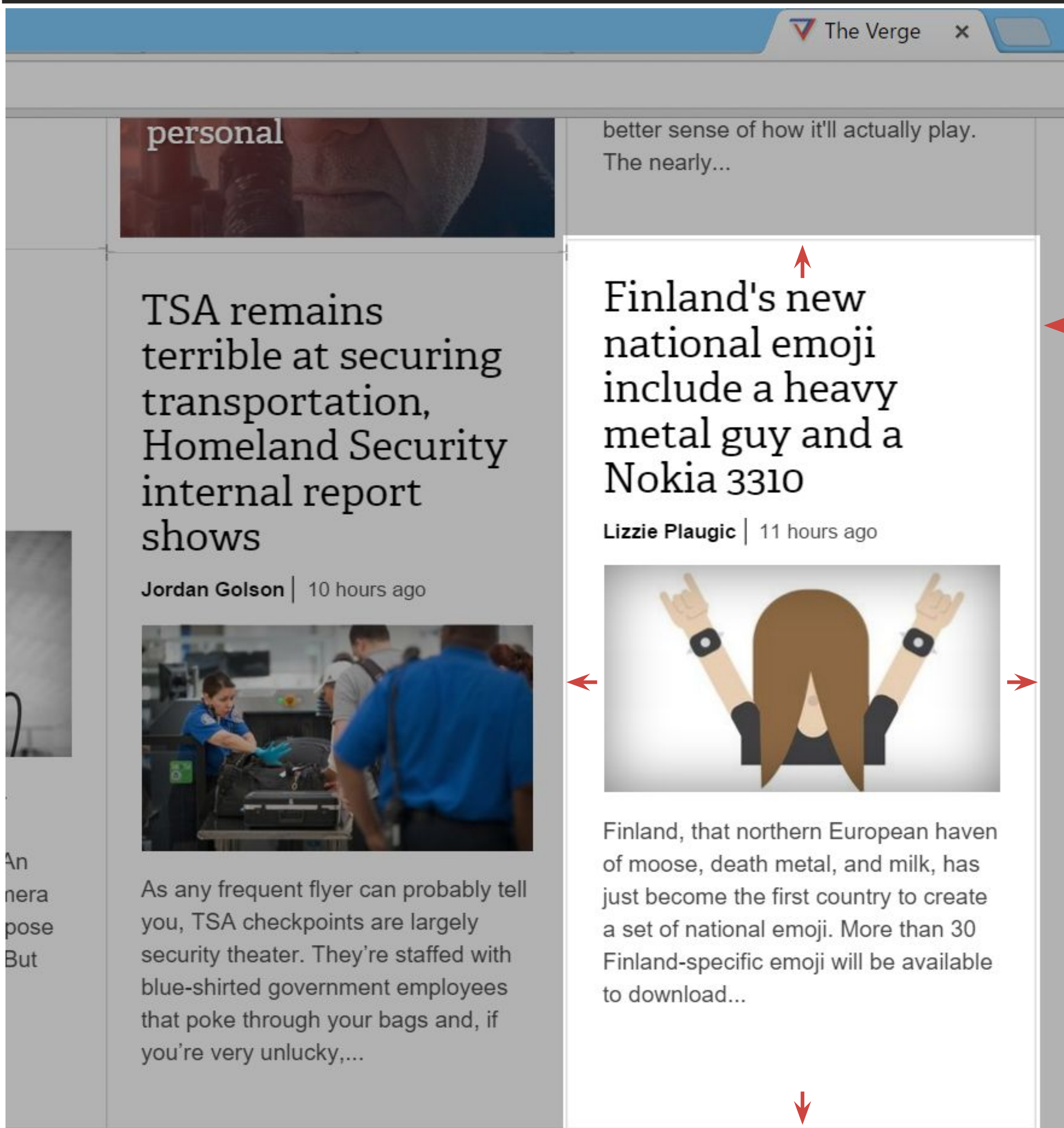
- **By id**

- Can only be used once within HTML.
- The HTML: `<form id="buyNow">...</form>`
- The CSS selector: `#buyNow { color: red }`

Layout: The Box Model (cont.)



Layout: The Box Model (cont.)



border: 1px;

padding: 19.5px;

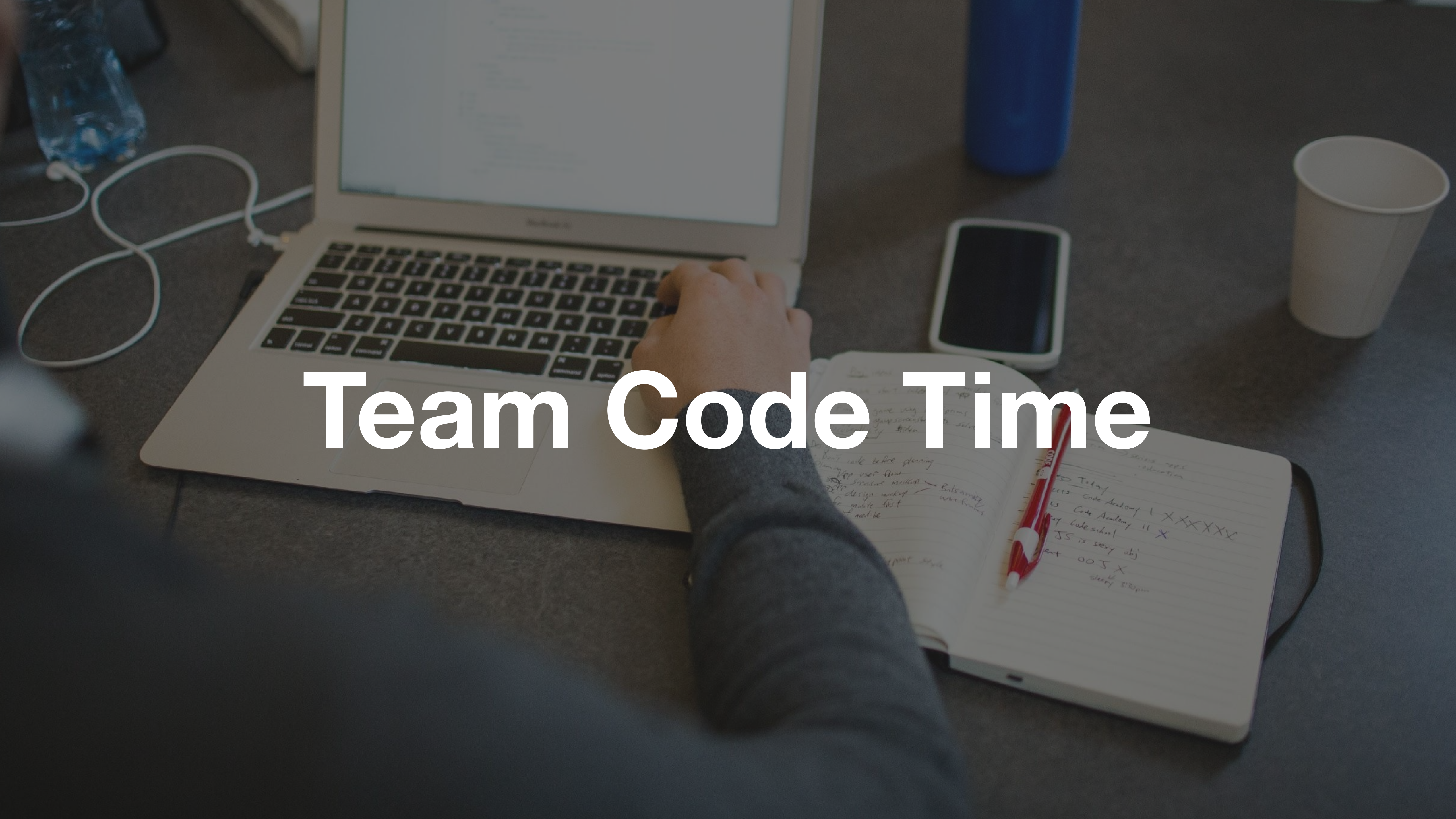
margin: 0px;

Layout: The Box Model

- It's the **padding**, **border**, and **margin** separating your content from other areas of content on your page.
- Use these properties wisely to create **negative space** to improve your layout and user experience (UX).
- Temporarily add this code to the top of your css file

```
* { outline: dotted 1px green; }
```
- Then add padding, border and margin properties.
- Do an ACP

Team Code Time



Style It!

- **Link one css file to each of your html files.**
- **Look at each of the styles and figure out which elements on the page they refer to**
- **Make some changes! Maybe you want to alter:**
 - **Text colors (color)**
 - **Background colors (background-color)**
 - **Link underlines (text-decoration)**
 - **Fonts (font-family)**

The Power Of CSS

- csszengarden.com
- In ① and ②, the HTML did not change. The CSS did.



What To Expect

- Those CSS Zen Garden examples? Those were created by advanced users of CSS. Your site might look more like this:



The image features a dark gray background with several thin, curved lines in red and white. These lines are positioned on the left and right sides, creating a sense of motion or a stylized frame. The word "JavaScript" is centered in a bold, white, sans-serif font.

JavaScript

JavaScript

- Tells the browser what it should **do** when certain things happen
 - The page loads
 - The user clicks on a button
 - A form is submitted
 - The page needs to fetch extra data

Username or Email Address

dahlbyk

Password

|

☒ Stay signed in

Log In

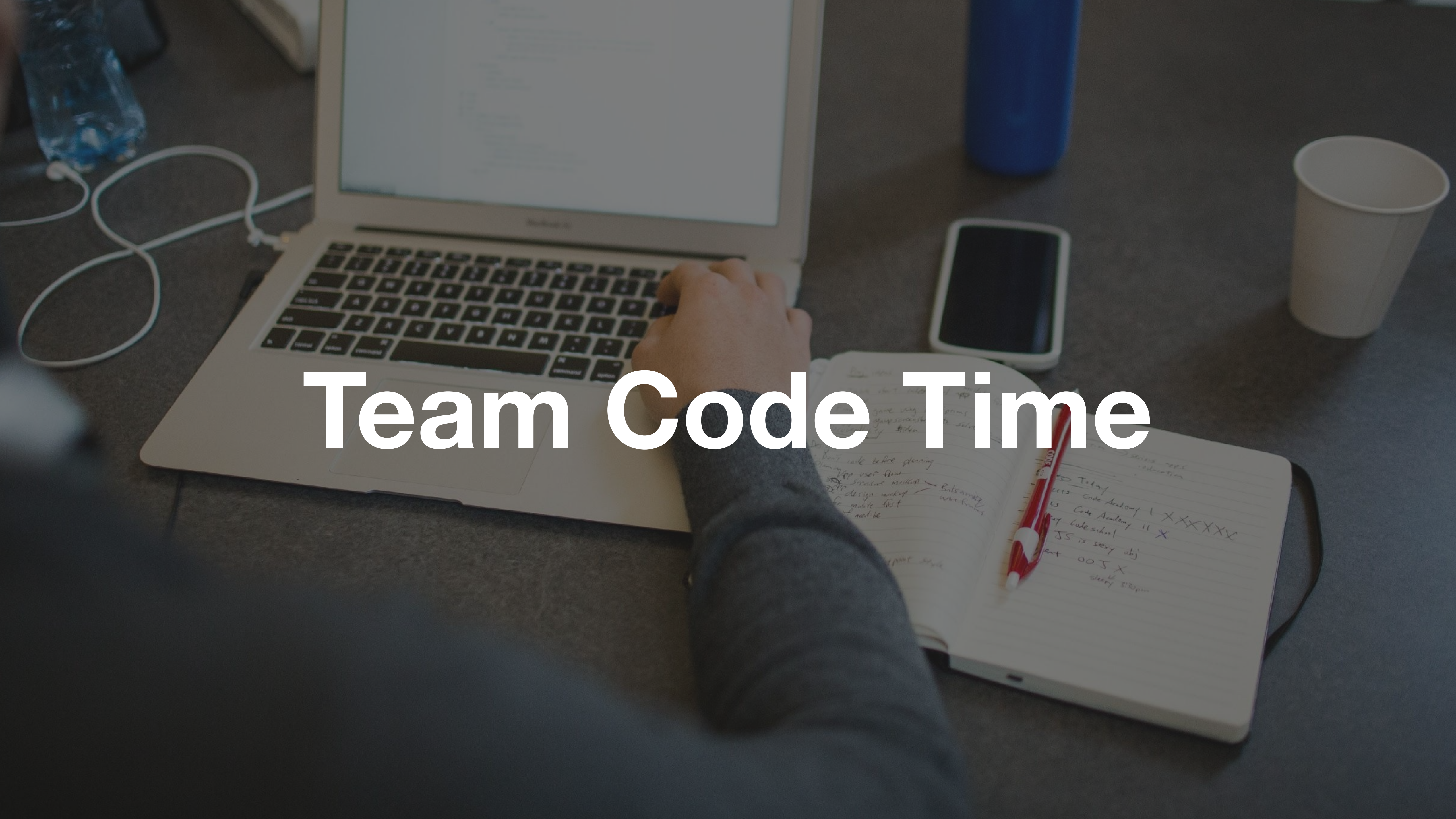
[Register](#)

[Lost your password?](#)

JavaScript - What It Looks Like (jQuery)

```
$(function() {  
    $("#donate").on("submit", function(e) {  
        e.preventDefault();  
  
        var amount = $("input[name=amount]").val();  
  
        if (amount) {  
            $.post("donate", {amount: amount},  
                function(res) {  
                    var donateRes = res.msg;  
                    $("#donateRes").text(donateRes);  
                });  
        }  
    });  
});
```


Team Code Time





the Developer Life

WHAT'S IN A NAME?

- People who write code for a living go by many different names:
 - Web developer
 - Software development engineer (SDE)
 - Front-end developer
 - Software engineer
 - UX Designer
- The names differ by stack, company, and where you spend most of your time (front end, back end, or both)

COMMONALITIES

- Regardless of where you work, developers usually:
 - Identify bugs or new features
 - Plan out how to fix the bug or make the new feature
 - Write code
 - Review code
 - Make sure code gets tested
 - Work with a team to design and implement solutions

DEVELOPMENT-RELATED JOBS

- Testing: Make sure what's been developed actually works.
- Sales engineer: Use your strong interpersonal skills to help potential buyers understand the technical side of your product.
- Web producer: Manage the content of a site using a CMS (content management system) and some HTML/CSS/JavaScript.
- Technical writer: Create accurate, well-written documentation that helps customers use your product.
- Project manager: Herd cats. Keep the end goal in mind.

BECOMING A DEV

DEVELOPMENT MIGHT BE A GOOD FIT FOR YOU IF...

- You want a field where you are constantly learning
- Learning languages appeals to you
- You like to finish a problem once you tackle it
- You want to work with interesting people, who will challenge you to think differently and get better at what you do
- You think solving puzzles is fun
- You want to help make things that change the world (or at least one small corner of it)

GETTING THERE: SELF-STUDY

Online, books, meetups

- Pros:

- Less expensive
- Go at your own pace and schedule

- Cons:

- Hard to get the depth of knowledge required to break into the industry
- Fewer contacts to help you get a job

GETTING THERE: SCHOOLS

Specifically, code school programs like DeltaV

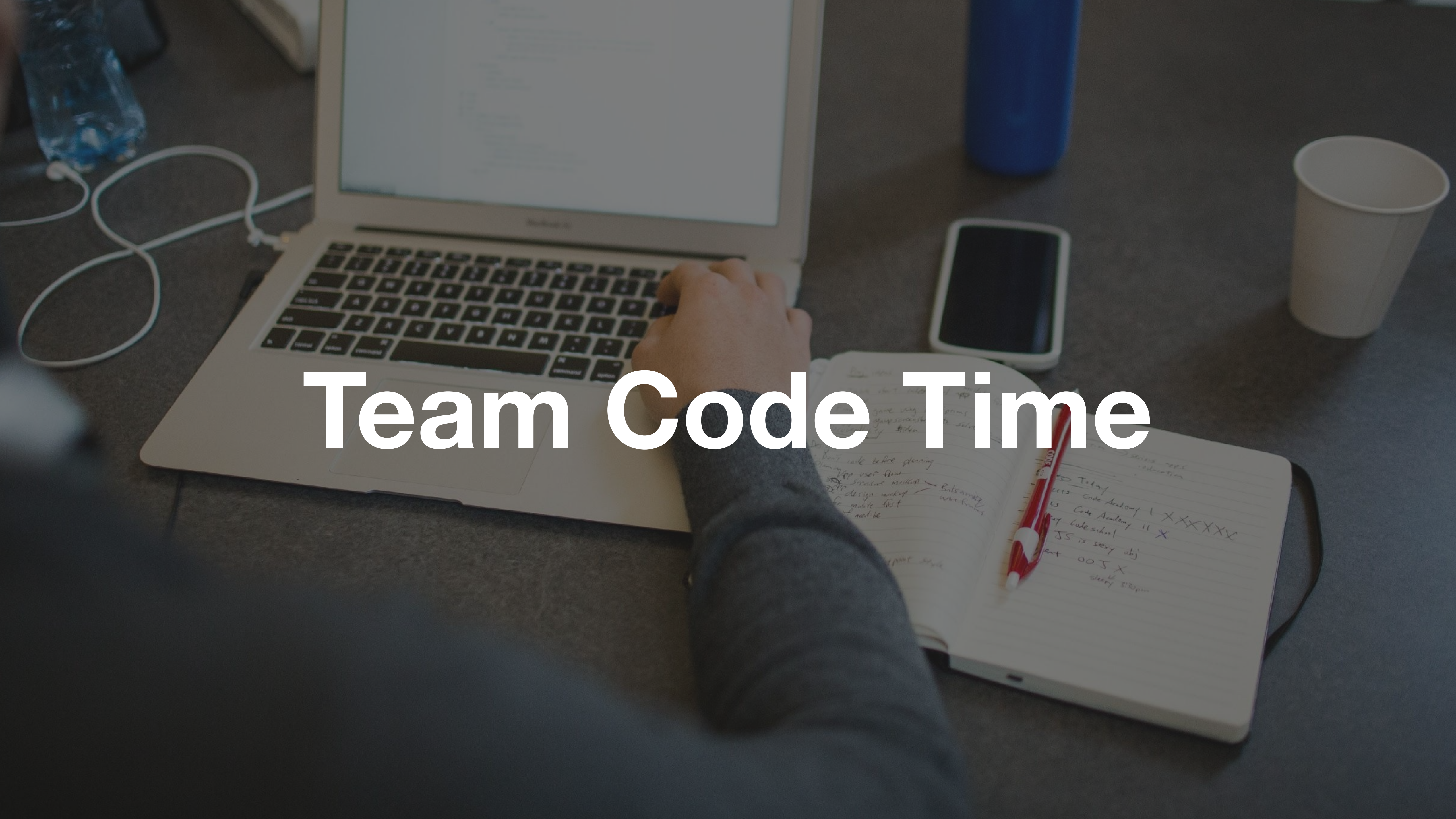
- **Pros:**

- Support from a community to meet your goals
- You get industry-ready, up-to-the-minute skills
- The school should actively support your job search

- **Cons:**

- It requires commitment and motivation to get the most out of it
- Expense (look for high placement rates and salaries for grads)

Team Code Time



FINISH IT!

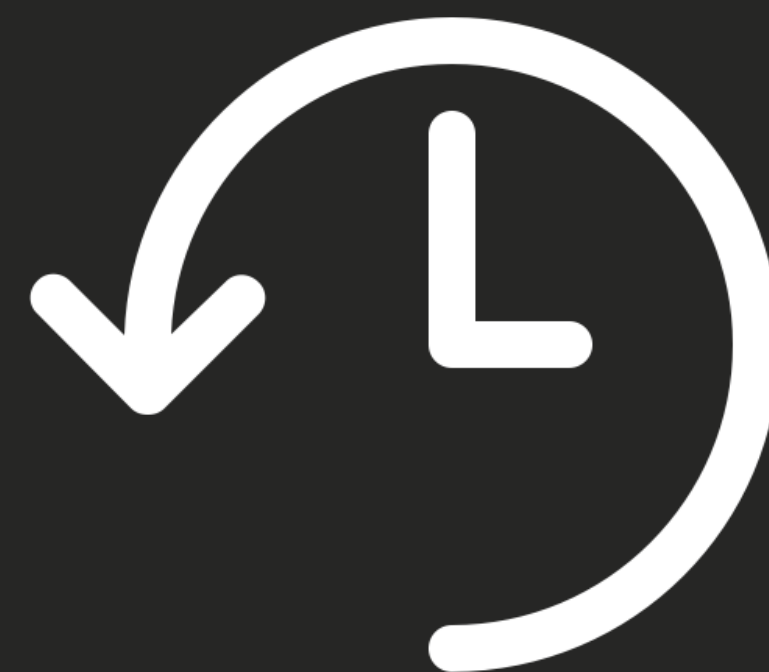
- **Each pair shares their HTML and CSS files on Slack.**
- **Individually, download all your team's files. Make sure you have everything.**
- **Test your site by opening one file in a browser. Your nav links should work now.**
- **If you have time left over, fine-tune the HTML and CSS.**

presentation!

~~Team Code Time~~

REFLECT

- **With your partner, spend a few minutes discussing:**
 - **Your favorite thing you learned today**
 - **What surprised you about today**
 - **Where you want to go next in your coding journey**



SURVEY

The image features a dark gray background with several thin, curved lines in red and white. These lines are positioned on the left and right sides, creating a sense of motion or a frame. The text "NEXT STEPS" is centered in the middle of the image in a bold, white, sans-serif font.

NEXT STEPS

WHERE TO LEARN NEXT: DeltaV

Congratulations on completing Code 101! You're about to enter an exciting industry and build some amazing things. So what's next in your journey?

If you liked this workshop, 102 or 201 is your next step.

- [Code 102](#) - Two Saturdays - Intro to JavaScript
- [Code 201](#) - 4 weeks, full-time
 - Enrollment open **now**, Class begins early June
 - [Start pre-work](#) early!

WHERE TO LEARN NEXT: ONLINE

Lots of great tutorials online for beginner-intermediate coders

- Codecademy ([HTML](#), [JavaScript](#))
- Treehouse
- Examples: <https://codepen.io/>

Books:

- The Jon Duckett books, which are our textbooks for 201, are an easy-to-understand reference for HTML, CSS, and JavaScript.

WHERE TO LEARN NEXT: MEETUPS

There are a ton of tech-related meetups.

- <http://techcorridor.io/> Slack / Twitter / Meetup
- <https://www.meetup.com/techcorridorio/>
 - Iowa JS
 - IC Game Dev
 - So much more...

The image features a dark gray background with several thin, curved lines in red and white. These lines are positioned on the left and right sides, creating a sense of motion or a frame. The word "CELEBRATE!" is centered in a bold, white, sans-serif font.

CELEBRATE!